

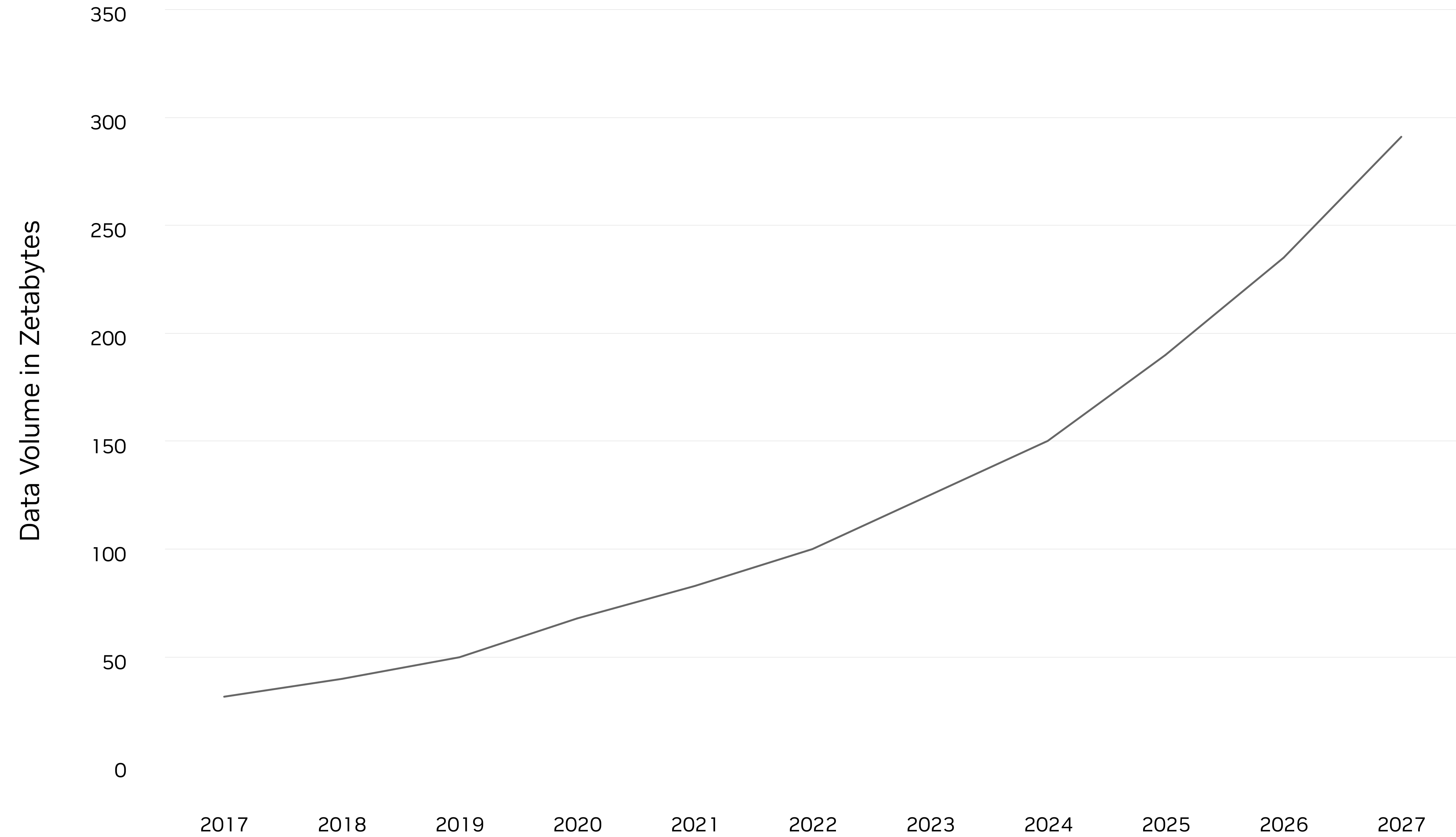


Getting Started with Multi-GPU Scaling: Distributed Libraries

Mads R. B. Kristensen, Principal Software Engineer, NVIDIA

Why move to multiple GPUs?

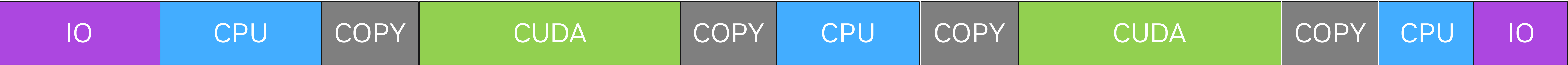
Data continues to Explode



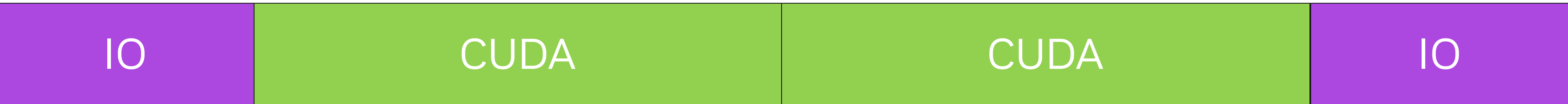
Source: IDC, Revelations in the Global DataSphere, US49346123, July 2023

History: the CUDA Ecosystem

Initially, we had individual single-GPU accelerated libraries.



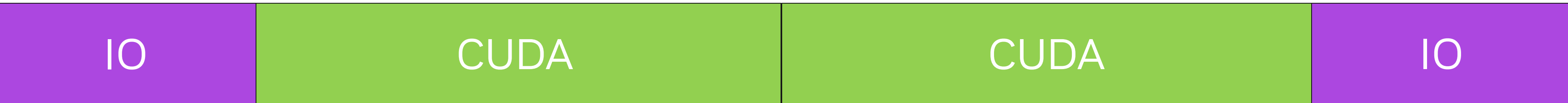
With an evolved single-GPU ecosystem, we now have memory compatible CUDA libraries.



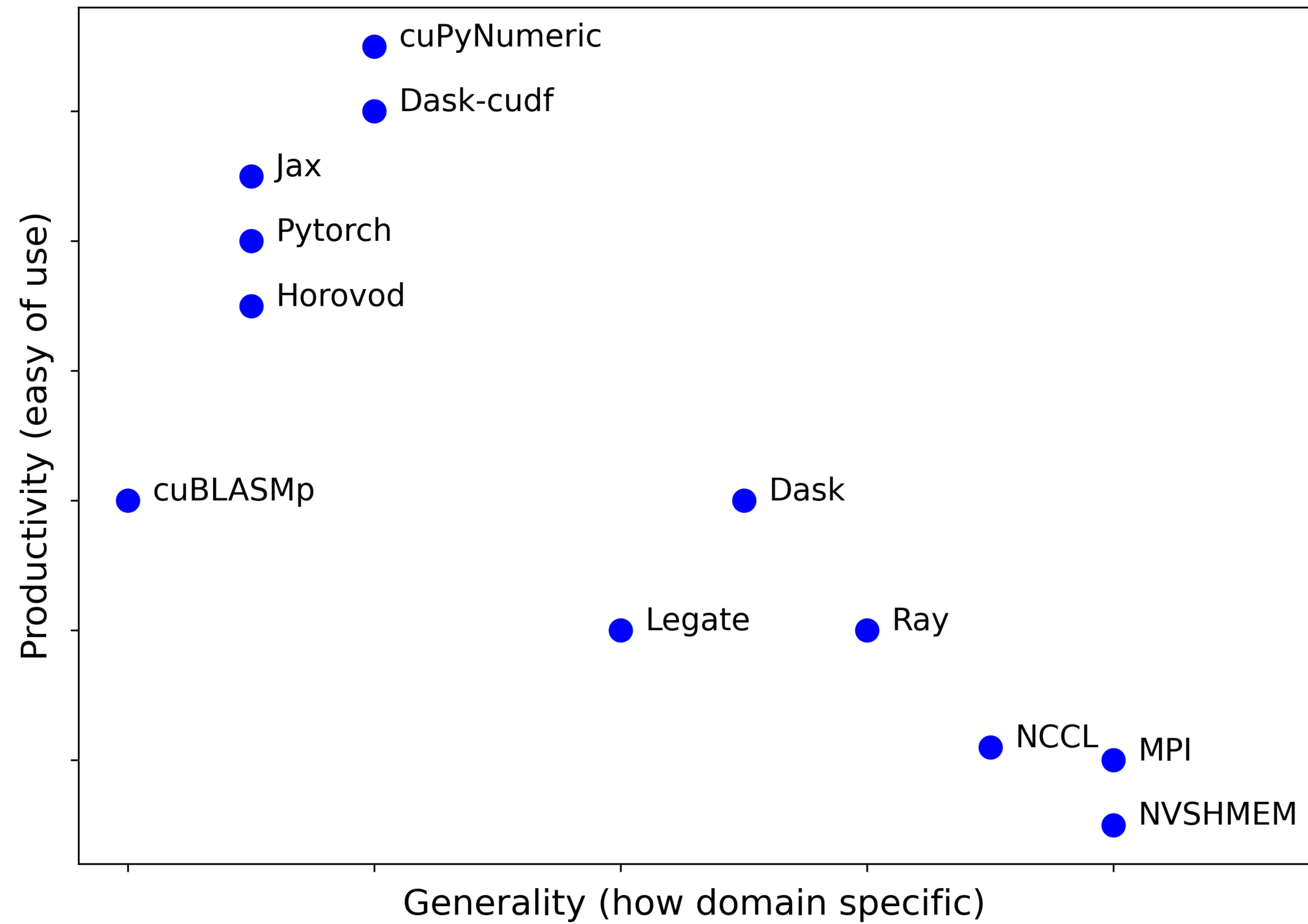
The multi-GPU ecosystem isn't as evolved.



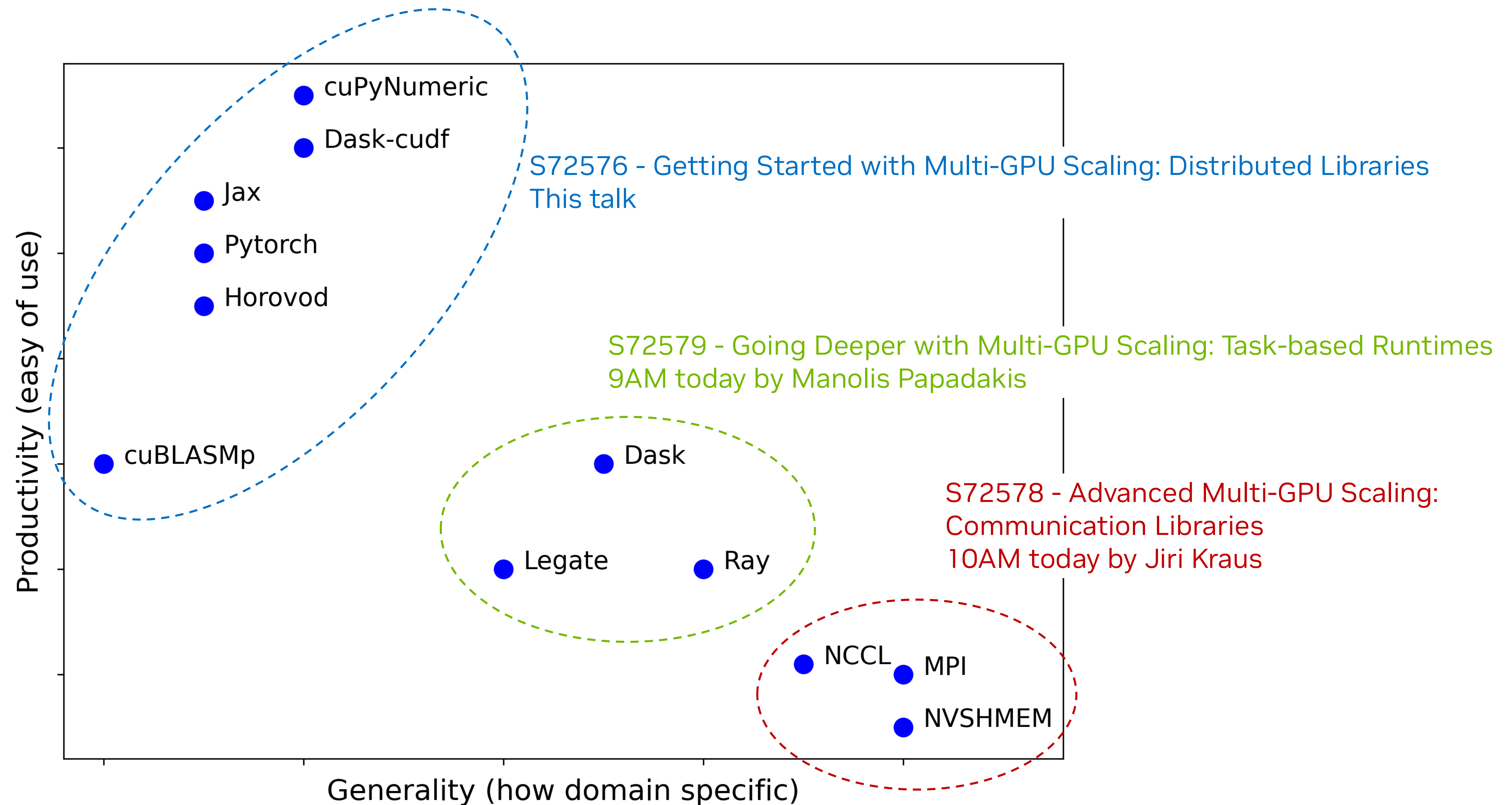
But we are developing unifying multi-GPU frameworks.



Multi-GPU Programming Landscape



Multi-GPU Programming Landscape



Data Parallelism

Explicit

E.g. cuBLASMP, PyTorch, NCCL, ...

- The user code runs on multiple machines such as SPMD.
- Communication through libraries such as MPI, UCX, or Sockets.
- Pros
 - Force users to think about data distribution and parallelization.
 - Compatible with other libraries that uses the same distributed model (huge ecosystem).
- Cons
 - Steep learning curve for users new to the specific distribution model.

Implicit

E.g. CuPyNumeric, Dask-cuDF, Legate, ...

- The user code (appears) to run on a single machine.
- Implicit communication through the computation library.
- Pros
 - Sequential control flow.
 - Easy to port existing sequential code base
 - Eliminate a broad range of parallelization bugs
 - By isolating the user from the parallelization, the framework has a lot of freedom when it comes to optimizations.
 - Inherent the ecosystem of the computation library.
- Cons
 - Harder to reason about performance.
 - A very cheap operation in a local execution might be very expensive in a distributed execution.
 - Requires a deep understanding of the library to implement user-defined operations.

MPI (Python)

```
from mpi4py import MPI
import cupy
import math

rank = comm.Get_rank()
worldsize = comm.Get_size()
grid_size = int(math.sqrt(worldsize))

# Create 2D Cartesian topology
cart_comm = comm.Create_cart(grid_size, ...)
coords = cart_comm.Get_coords(rank)

# Initial alignment
left_rank = cart_comm.Shift(1, -coords[0])[0]
up_rank = cart_comm.Shift(0, -coords[1])[1]
cart_comm.Sendrecv_replace(A_block, ...)
cart_comm.Sendrecv_replace(B_block, ...)

# Initialize result block
C_block = cupy.zeros((block_size, block_size))

# Main computation loop (Cannon's algorithm)
for i in range(grid_size):
    C_block += cupy.matmul(A_block, B_block)

    # Shift A left and B up
    left_rank = cart_comm.Shift(1, -1)[0]
    up_rank = cart_comm.Shift(0, -1)[1]
    cart_comm.Sendrecv_replace(
        A_block, left_rank, ...
    )
    cart_comm.Sendrecv_replace(
        B_block, left_rank, ...
    )
...
```

`mpirun -np <n ranks> python matmul.py`

Explicit Data Parallelism

Distributed Matrix Multiplication

CuBLASMp (C)

```
cal_comm_create_mpi(
    MPI_COMM_WORLD, rank, nranks, local_device,
    &comm
);

cublasMpGridCreate(..., cal_comm, &grid);

cublasMpMatrixDescriptorCreate(..., grid, &desc);

cublasMpCreate(&handle, stream)

cublasMpMatmul_bufferSize(
    handle,
    ...
    &workspaceInBytesOnDevice,
    &workspaceInBytesOnHost);

cublasMpMatmul(
    handle,
    ...,
    d_work,
    workspaceInBytesOnDevice,
    h_work,
    workspaceInBytesOnHost
);
```

`mpirun -np <n ranks> ./matmul`

PyTorch Distributed (Python)

```
import torch
import torch.distributed as dist

torch.cuda.set_device(
    int(os.environ["LOCAL_RANK"]))
)
dist.init_process_group("nccl")
rank = dist.get_rank()
worldsize = dist.get_world_size()
mesh = DeviceMesh("cuda", ...)

a = DTensor.rand(mesh, ...)
b = DTensor.rand(mesh, ...)
result = torch.matmul(a, b)
```

`mpirun -np <n ranks> python matmul.py`
`torchrun --nnodes=<n ranks> matmul.py`



Implicit Data Parallelism

- CuPyNumeric

CuPyNumeric

Seamlessly scaling of NumPy code

```
import cupynumeric as np

x = np.zeros_like(b)
r = b - A.dot(x)
p = r
rsold = r.dot(r)
max_iters = b.shape[0]

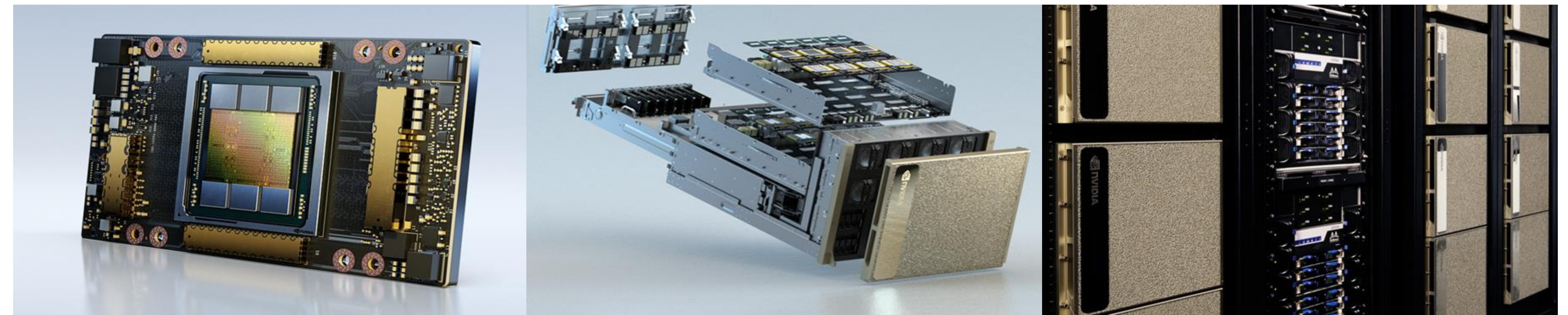
for i in range(max_iters):
    Ap = A.dot(p)
    alpha = rsold / (p.dot(Ap))
    x = x + alpha * p
    r = r - alpha * Ap
    rsnew = r.dot(r)

    if np.sqrt(rsnew) < tolerance:
        break

beta = rsnew / rsold
p = r + beta * p
rsold = rsnew
```

Conjugate Gradient Solver

- Infer data parallelism from NumPy array operations.
- Sequential user-code.
- Lazy Evaluation.
- Control flow duplication.

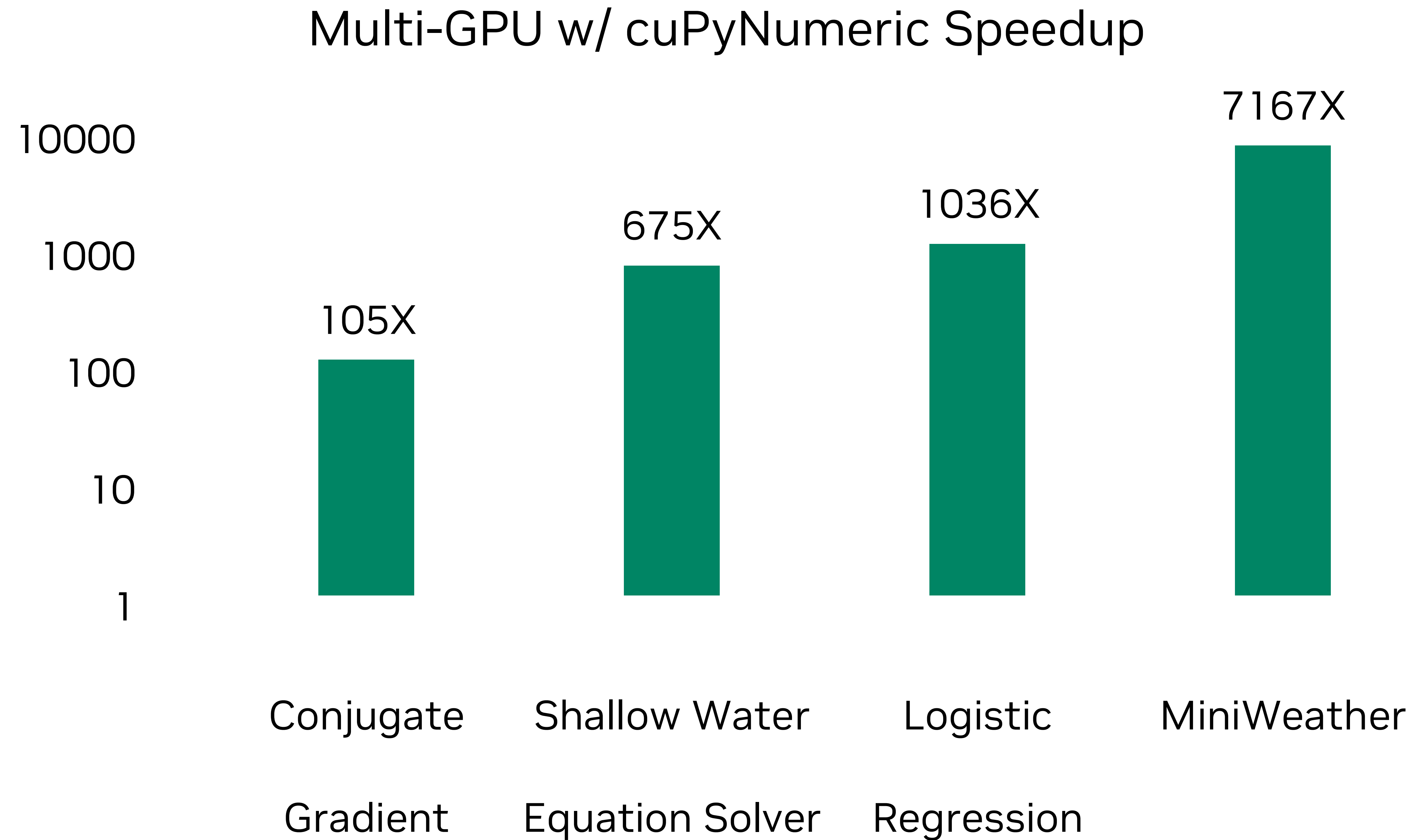


Single
GPU

Single-Node
Multi-GPU

Cloud &
Supercomputer

A Not So Fair Comparison



- It's unfair because the number is single CPU vs 8 GPUs (DGX-H100).
- It's also workload dependent.
- In the real-world use cases the speedup will be less end to end.
- It's still relevant because:
 - The runs are same code w/o any change.
 - NumPy can only one CPU and cuPyNumeric scales.

```
>>> import cupynumeric as np
>>> # Create two random vectors distributed on multiple GPUs.
... vec1 = np.random.rand(1_000_000)
... vec2 = np.random.rand(1_000_000)

>>> # cuPyNumeric ndarray.
... type(vec1)
<class 'cupynumeric._array.array.ndarray'>

>>> # Compute element-wise addition.
... res = vec1 + vec2

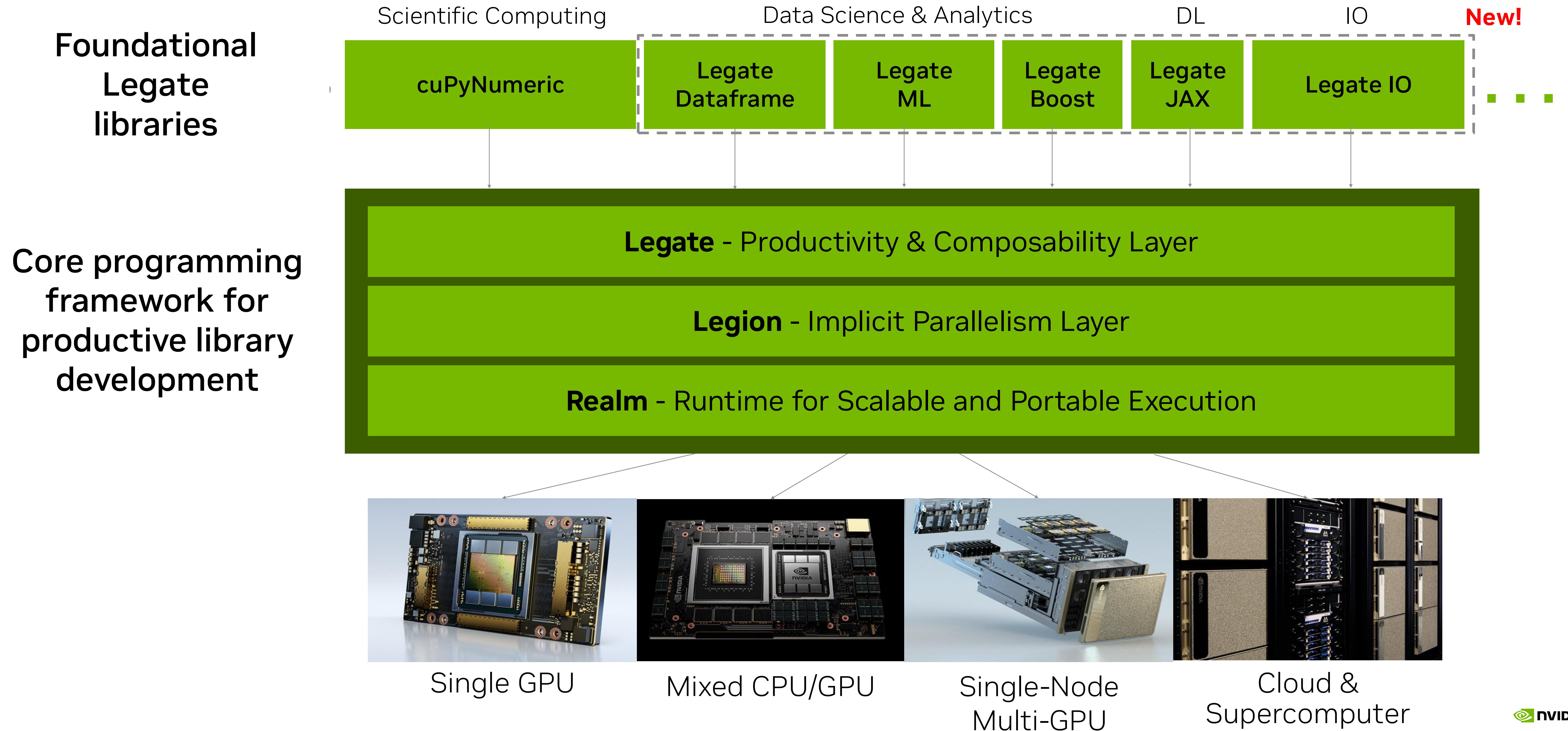
>>> # No computation needed, just metadata.
... res.shape
(1000000,)

>>> # Do the computation, move the result to the frontend,
... # and print to screen.
... print(res.sum())
999748.3077385854
```

legate --gpus <no-gpus> script.py

Legate Ecosystem

Composable and accelerated libraries for scientific computing and beyond





Implicit Data Parallelism

- **Dask-cuDF**

Dask-cuDF

Accelerate and distribute Pandas

```
import dask
import dask.dataframe as dd

# Set the default backend to use GPUs
dask.config.set({"dataframe.backend": "cudf"})

# US house prices by state
data =
https://www.fhfa.gov/hpi/download/quarterly\_datasets/hpi\_at\_state.c
sv

# Load the dataset
df = dd.read_csv(
    data, names=["state", "year", "quarter", "price"]
)

# Compute mean price by state
mean_prices =
df.groupby("state")["price"].mean().compute()
print(mean_prices)
```

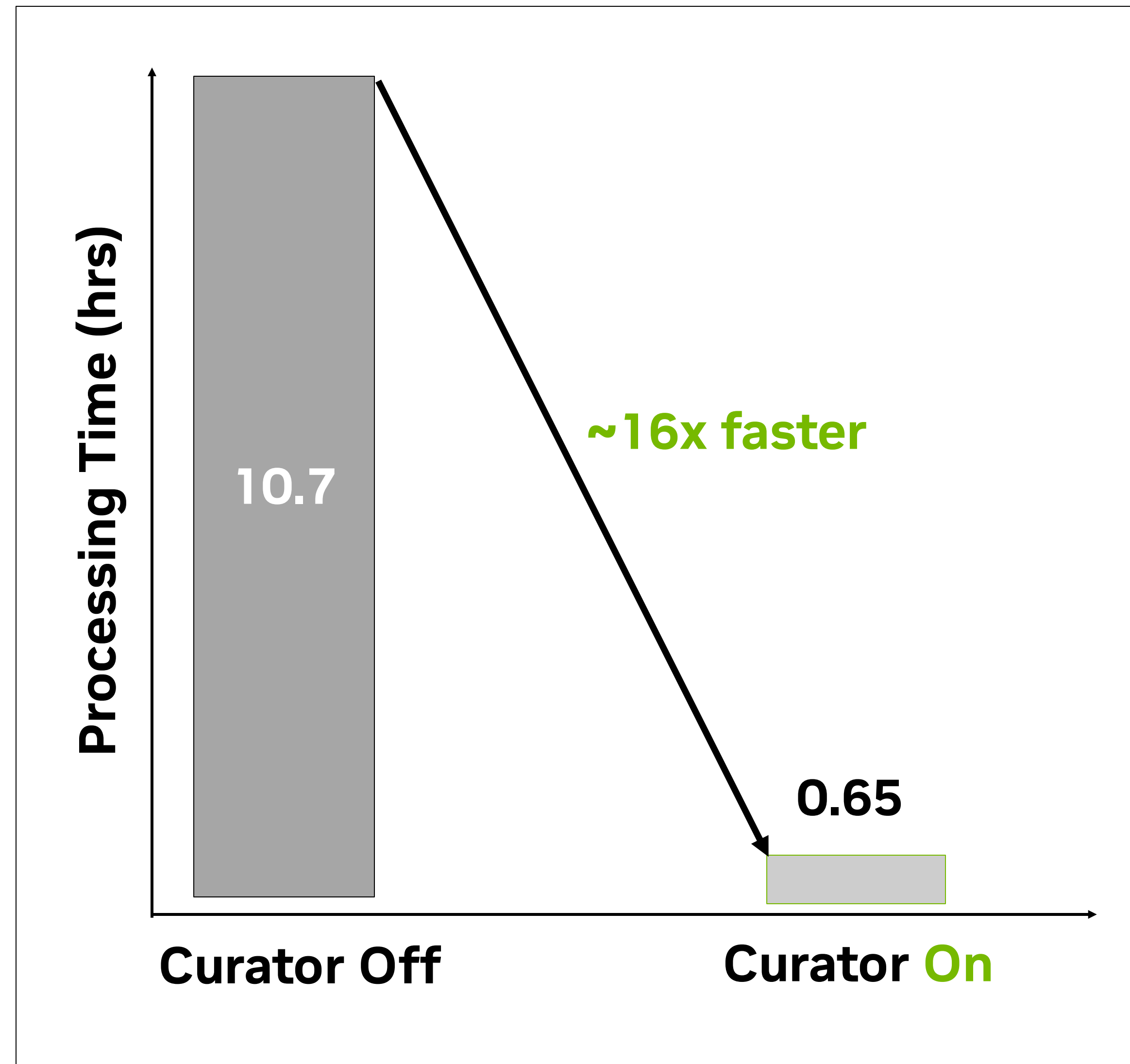
US House Prices

- Infer data parallelism from DataFrame operations.
- Sequential user-code.
- Lazy Evaluation.
- Memory Spilling (GPU->CPU->Disk)

Another Not So Fair Comparison

NeMo Curator

~16x Faster Processing

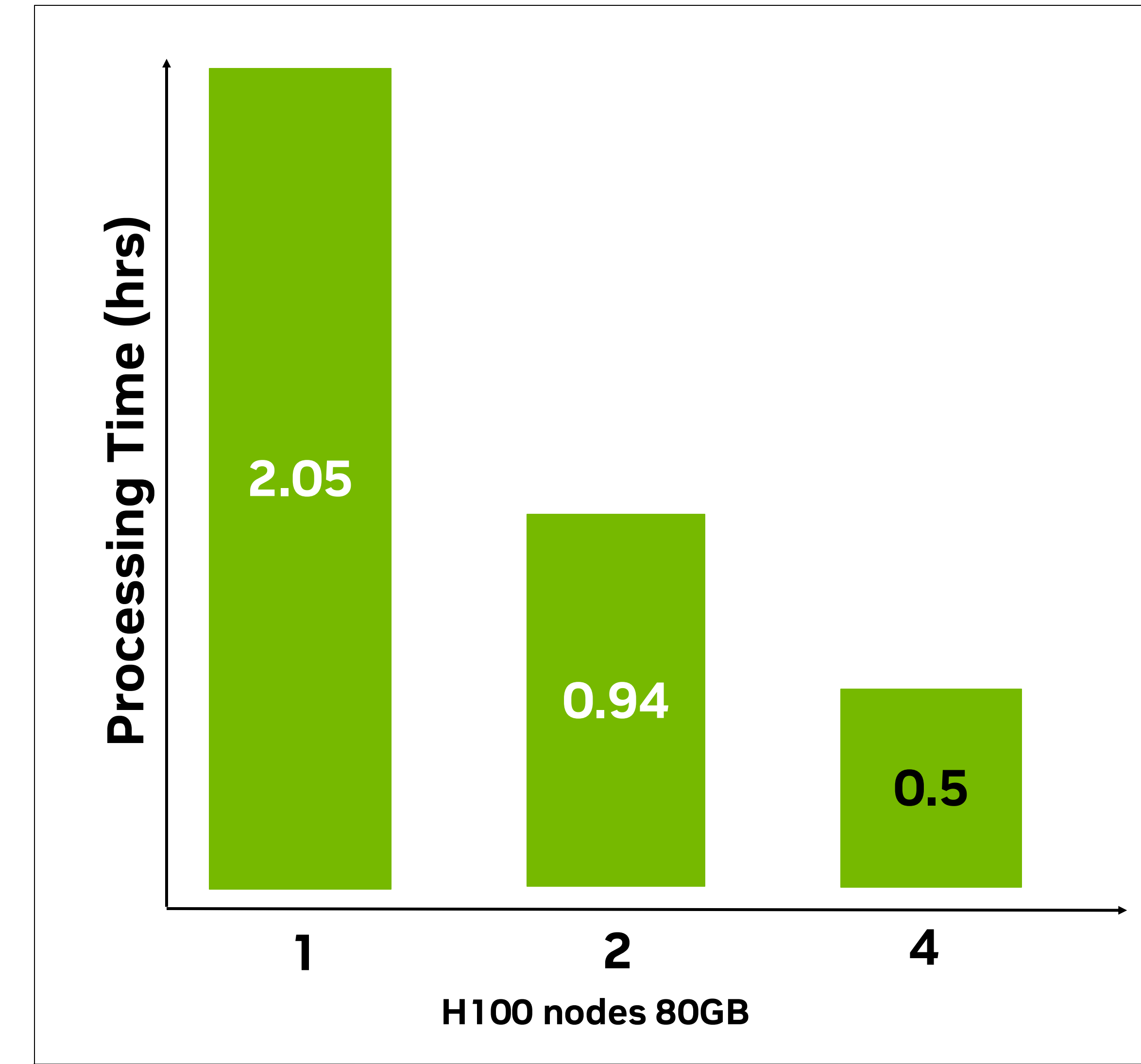


Processing time for fuzzy deduplication of RedPajama-v2 subset (8TB/1.78T tokens)

On: Data processed with NeMo Curator on 3 H100 nodes

Off : Data processed with a leading alternative library on CPUs

Near Linear Scaling



Processing time for fuzzy deduplication of RedPajama-v2 subset (8TB/1.78T tokens)

Scaling on 1, 2, 3, 4 H100 nodes 80GB

Dask and RAPIDS

Drop-In Acceleration for DataFrames, SQL, and Machine Learning

CPU

Dask DataFrame

```
>>> import dask.dataframe as dd

>>> df = dd.read_parquet("file")
>>> df.groupby("col").mean()
>>> df.rolling(window=3).sum()
```

Dask SQL

```
>>> from dask_sql import Context
>>> c = Context()
>>> c.create_table("timeseries", df)

>>> result = c.sql("""
SELECT name, SUM(x) as "sum"
FROM timeseries
GROUP BY name
""")
```

Dask XGBoost

```
>>> import xgboost as xgb

>>> dtrain = xgb.dask.DaskDMatrix(...)
>>> output = xgb.dask.train(...,
{'tree_method': "hist"})
>>> prediction = xgb.dask.predict(client,
output, dtrain)
```

GPU

Dask DataFrame

```
>>> import dask.dataframe as dd
>>> import dask
>>> dask.config.set(
    {"dataframe.backend": "cudf"}
)
>>> df = dd.read_parquet("file")
>>> df.groupby("col").mean()
>>> df.rolling(window=3).sum()
```

Average Speedup: 10-20x

Dask SQL

```
>>> from dask_sql import Context
>>> c = Context()
>>> c.create_table(
    "timeseries", df, gpu=True
)
>>> result = c.sql("""
SELECT name, SUM(x) as "sum"
FROM timeseries
GROUP BY name
""")
```

Average Speedup: 10-20x

Dask XGBoost

```
>>> import xgboost as xgb

>>> dtrain = xgb.dask.DaskDMatrix(...)
>>> output = xgb.dask.train(...,
{'tree_method': "gpu_hist"})
>>> prediction = xgb.dask.predict(client,
output, dtrain)
```

Average Speedup: Up to 20x



Not Perfect

Sometimes you must take the control

Common Pitfalls

AVOID loops with indexing

- Use of non-distributed operations.
 - E.g., use Python loops to iteration over data.
- Performance depend on the underlying data distribution.
 - E.g., matrix transpose might be *cheap* on a single node but very *expensive* on multiple nodes.
- Accidentally gathering everything on a single node.

```
# x and y are three-dimensional arrays

# Not recommended: Naive element-wise implementation
for i in range(ny):
    for j in range(nx):
        x[0, j, i] = y[3, j, i]

# Recommended: Array-based implementation
x[0] = y[3]
```

Watch out for third-party libraries

```
import cupynumeric as np
import scipy.linalg

# m is a squared matrix

# Not recommended: use of non-legate library
scipy.linalg.cholesky(m)

# Recommended: use an equivalent legate library
np.linalg.cholesky(m)
```

AVOID advanced indexing

```
import cupynumeric as np

# Not recommended: don't use nonzero to get indices
indices = np.nonzero(h < 0)
x[indices] = y[indices]

# Recommended: Use boolean mask to update the array
cond = h < 0
x[cond] = y[cond]
```

Escape hatch

cuPyNumeric

What to do when cuPyNumeric doesn't support what you want?

- Use cuPyNumeric as a regular CuPy array.
 - Very easy.
 - Compatible with third party CuPy/NumPy libraries.
 - Very performance expensive!
 - All data must fit a single GPU.
- Write custom legate task.
 - Not as easy and requires more work.
 - Good performance.

```
import cupynumeric as np
import cupy

vec1 = np.random.rand(1_000_000)
vec2 = np.random.rand(1_000_000)
vec3 = vec1 + vec2

# Copy to a single GPU
local_vec = cupy.asarray(vec3)
res = local_vec + 42 # Local CuPy operation
print(res.sum())
```

Escape hatch

cuPyNumeric

What to do when cuPyNumeric doesn't support what you want?

- Use cuPyNumeric as a regular CuPy array.
 - Very easy.
 - Compatible with third party CuPy/NumPy libraries.
 - Very performance expensive!
 - All data must fit a single GPU.
- Write custom legate task.
 - Not as easy and requires more work.
 - Good performance.

```
import cupy
import cupynumeric as np
from legate.core import VariantCode
from legate.core.task import task, InputArray, OutputArray

vec1 = np.random.rand(1_000_000)
vec2 = np.random.rand(1_000_000)
vec3 = vec1 + vec2
res = np.empty_like(vec3)
my_task(vec3, res)
print(res.sum())

@task(variants=(VariantCode.GPU,), throws_exception=True)
def my_task(ary: InputArray, out: OutputArray) -> None:
    ary = cupy.asarray(ary)
    out = cupy.asarray(out)
    out[:] = ary + 42 # Local CuPy operation
```

Escape hatch

Dask-cuDF

Fallback to a single GPU

```
import dask.dataframe as dd

data_url = "https://www.fhfa.gov/.../hpi_at_state.csv"
df = dd.read_csv(
    data_url, names=["state", "year", "quarter", "price"]
)
local_df = df.compute() # Copy to a single GPU
res = local_df["year"] + 1
print(res.sum())
```

User task

```
import dask.dataframe as dd

data_url = "https://www.fhfa.gov/.../hpi_at_state.csv"
df = dd.read_csv(
    data_url, names=["state", "year", "quarter", "price"]
)
res = df.map_partitions(my_task)
print(res.sum().compute())

def my_task(local_df: cudf.DataFrame) -> cudf.DataFrame:
    return local_df["year"] + 1
```

Summary

What framework to use?

Good news, they are not mutually exclusive!
Start from the highest abstraction.

Explicit Data Parallelism

E.g. cuBLASMP, PyTorch, NCCL, ...

- Already knows MPI or similar SPMD framework.
- Interoperability with existing code base.
- Highest performance.

Implicit Data Parallelism

E.g. CuPyNumeric, Dask-cuDF, Legate, ...

- High productivity.
- Easy to port sequential code.
- Prototyping.
- Abstract domain from parallelization expertise.

CuPyNumeric Examples:

https://github.com/NVIDIA/accelerated-computing-hub/blob/main/Accelerated_Python_User_Guide/notebooks/Chapter_X_Distributed_Computing_cuPyNumeric.ipynb

Dask-cuDF Examples:

https://docs.rapids.ai/api/cudf/nightly/user_guide/10min/

CUDA Developer Sessions

General CUDA

- [S72571](#) - What's CUDA All About Anyways?
- [S72897](#) - How To Write A CUDA Program: The Parallel Programming Edition

CUDA Python

- [S72450](#) - Accelerated Python: Tour of the Community and Ecosystem
- [S72448](#) - The CUDA Python Developer's Toolbox
- [S72449](#) - 1001 Ways to Write CUDA Kernels in Python
- [S74639](#) - Enable Tensor Core Programming in Python With CuTe

CUDA C++

- [S72574](#) - Building CUDA Software at the Speed-of-Light
- [S72572](#) - The CUDA C++ Developer's Toolbox
- [S72575](#) - How You Should Write a CUDA C++ Kernel

Developer Tools

- [S72527](#) - It's Easier than You Think – Debugging and Optimizing CUDA with Intelligent Developer Tools

Connect with the Experts

- [CWE72433](#) - CUDA Developer Best Practices
- [CWE73310](#) - Using NVIDIA CUDA Compiler Tool Chain for Productive GPGPU Programming
- [CWE72393](#) - What's in Your Developer Toolbox? CUDA and Graphics Profiling, Optimization, and Debugging Tools

Multi-GPU Programming

- [S72576](#) - Getting Started with Multi-GPU Scaling: Distributed Libraries
- [S72579](#) - Going Deeper with Multi-GPU Scaling: Task-based Runtimes
- [S72578](#) - Advanced Multi-GPU Scaling: Communication Libraries

Performance Optimization

- [S72683](#) - CUDA Techniques to Maximize Memory Bandwidth and Hide Latency
- [S72685](#) - CUDA Techniques to Maximize Compute and Instruction Throughput
- [S72686](#) - CUDA Techniques to Maximize Concurrency and System Utilization
- [S72687](#) - Get the Most Performance from Grace Hopper



nvidia.com/gtc/sessions/cuda-developer